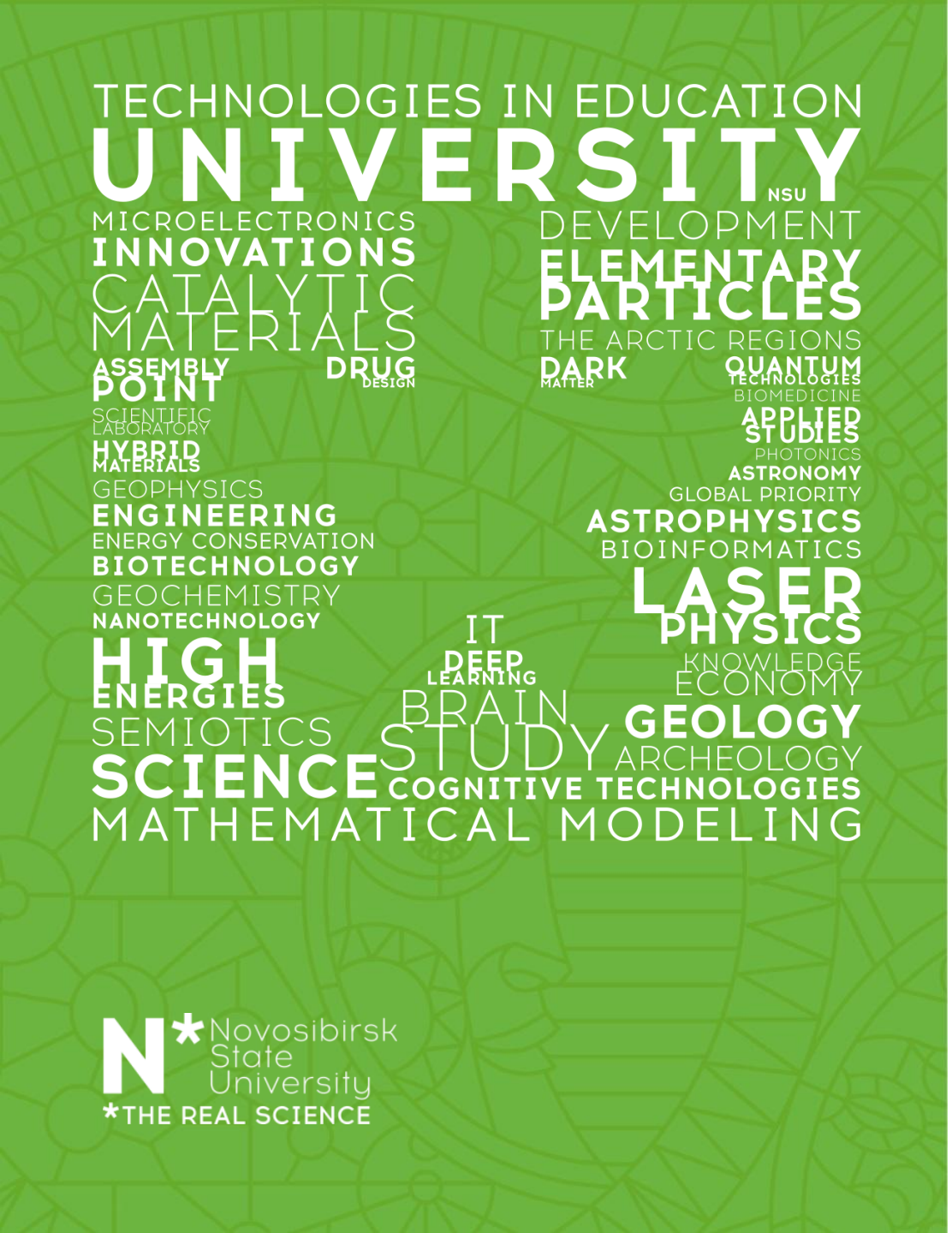


Машинное обучение
Семинар 9

Глушенко Андрей Валерьевич
Институт Интеллектуальной Робототехники

Практические аспекты

Метод опорных векторов

Разделение на обучающую, валидационную и тестовую выборки

Кросс-валидация

Метрики

Метод опорных векторов (SVM)

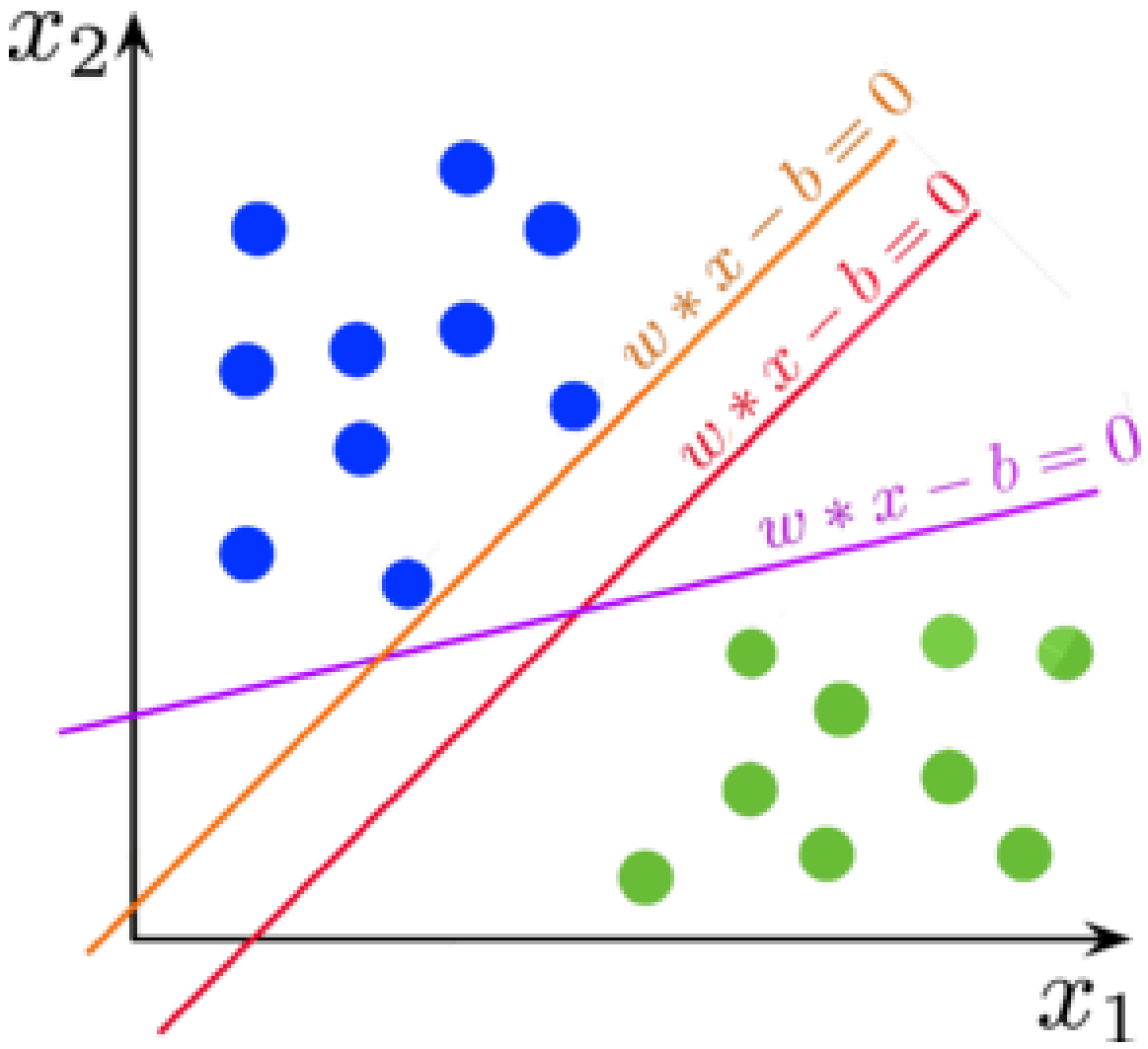
Метод опорных векторов (Support Vector Machine или **SVM**) — это один из самых популярных и эффективных алгоритмов обучения «с учителем», который используется для задач **классификации** и регрессии.

SVM. Основная идея

Представьте, что на плоскости перемешаны точки двух цветов. Задача SVM — провести между ними линию (или гиперплоскость в многомерном пространстве) так, чтобы разделить их «наилучшим» образом.

«Наилучшим» в данном случае считается такое разделение, при котором зазор (margin) между классами будет максимальным. Чем шире эта «пустая дорога» между группами данных, тем увереннее алгоритм будет классифицировать новые объекты.

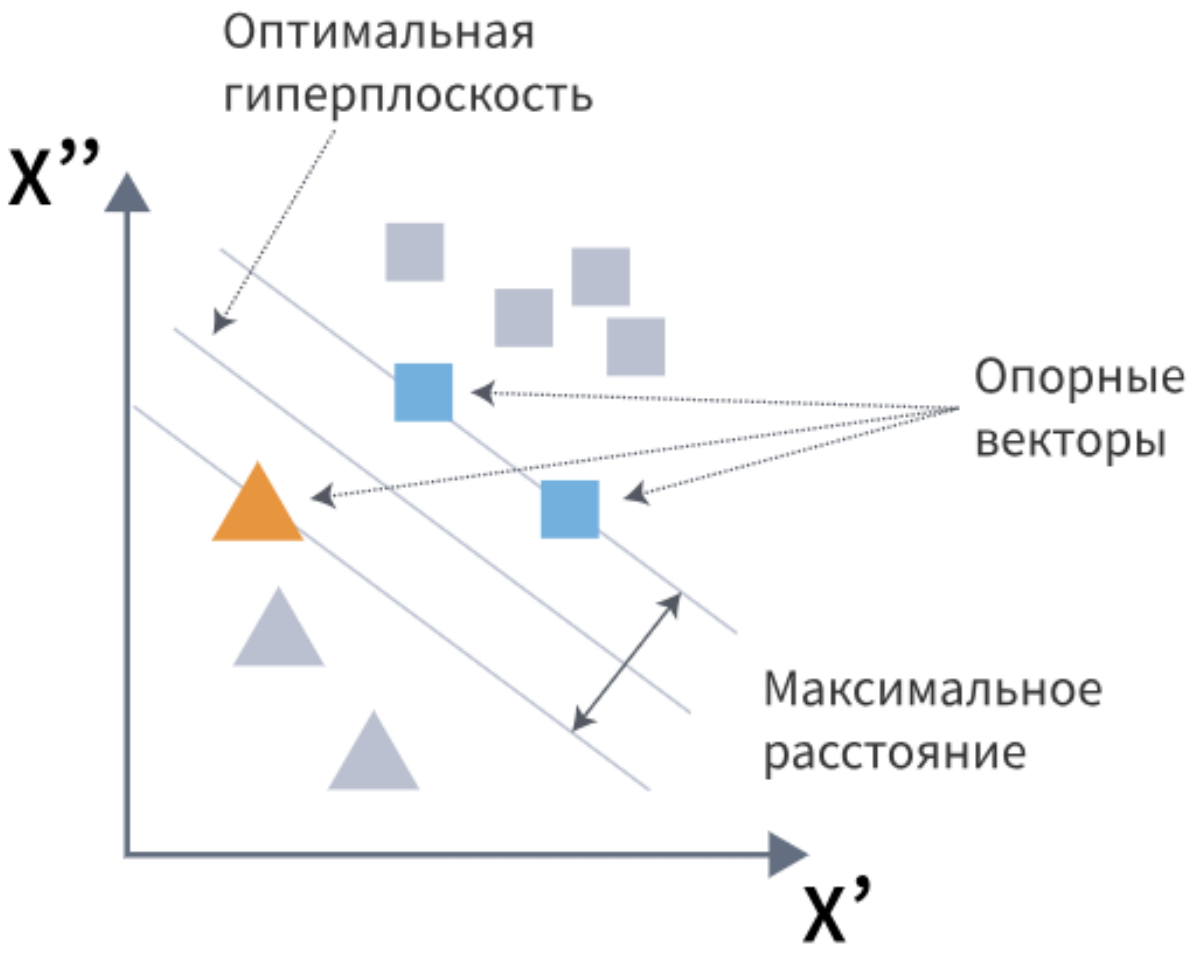
SVM. Основная идея



Основные понятия

- **Опорные векторы:** Это «крайние» точки в наборах данных, которые лежат ближе всего к разделяющей линии. Именно они определяют положение границы. Если их убрать, граница сдвинется, поэтому они и называются опорными.
- **Гиперплоскость:** Это сама граница раздела. В 2D — это линия, в 3D — плоскость, а в более высоких размерностях — гиперплоскость.
- **Ядерный трюк (Kernel Trick):** Если данные невозможно разделить прямой линией (например, один класс внутри другого), SVM переносит их в пространство более высокой размерности, где разделение становится возможным.

Основные понятия



Преимущества

- **Точность:** SVM отлично работает на выборках с четкой границей разделения.
- **Эффективность:** Он справляется даже тогда, когда количество признаков (колонок в таблице) больше, чем количество самих объектов.
- **Устойчивость:** Благодаря фокусу на зазоре, метод менее склонен к ошибкам из-за случайных выбросов, которые находятся далеко от границы.

Алгоритм SVM

1. Поиск разделяющей гиперплоскости

Сначала алгоритм ищет все возможные варианты границ (линий или плоскостей), которые могут разделить ваши данные на два класса.

В n-мерном пространстве разделяющая граница описывается линейным уравнением:

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = \boldsymbol{w}^{\boldsymbol{T}}\boldsymbol{x} + \boldsymbol{b} = \boldsymbol{0}, \text{где:}$$

x — вектор признаков объекта.

w — вектор весов (нормаль к гиперплоскости), определяющий её ориентацию.

b — смещение (bias), определяющее расстояние от начала координат.

Алгоритм SVM

Условие классификации

Для каждого объекта x_i из обучающей выборки мы хотим, чтобы алгоритм выдавал правильную метку y_i(принимает значения +1 или -1):

$$w^T x_i + b \ \geq 1, \text{если} y_i = +1$$
$$w^T x_i + b \ \leq 1, \text{если} y_i = -1$$

Это можно объединить в одно неравенство:

$$y_i(w^T x_i + b) \ \geq 1$$

Алгоритм SVM

2. Максимизация зазора (Geometric Margin)

Из всех найденных границ SVM выбирает ту, у которой самый широкий зазор.

- **Зазор** — это расстояние от границы до ближайших точек каждого класса.
- Алгоритм стремится сделать эту «дорогу» максимально широкой, чтобы минимизировать риск ошибки при появлении новых данных.

Геометрическое расстояние от точки до гиперплоскости равно $\frac{1}{\|w\|}$. Общий зазор между двумя классами равен $\frac{2}{\|w\|}$.

Чтобы сделать зазор максимальным, нам нужно минимизировать норму вектора весов $\|w\|$.

Математически задача оптимизации выглядит так:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \text{ для всех } i \text{ таких, что } y_i(w^T x_i + b) \geq 1$$

Алгоритм SVM

3. Определение опорных векторов

В процессе выбора границы алгоритм находит те самые точки, которые стоят вплотную к «обочине» зазора. Это и есть опорные векторы.

- Все остальные точки данных, которые находятся дальше, алгоритму не важны — они не влияют на положение границы.

Алгоритм SVM

4. Мягкий зазор (Soft Margin)

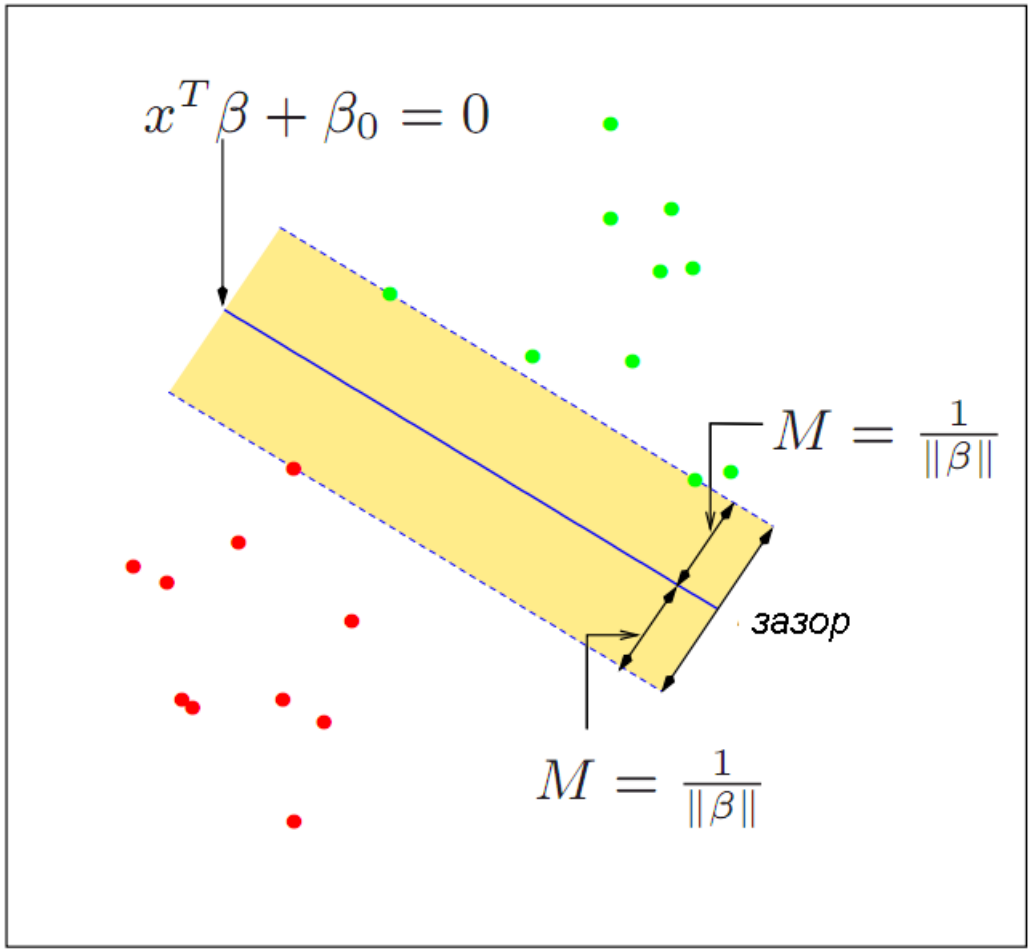
В реальности данные редко разделяются идеально. Чтобы позволить алгоритму допускать небольшие ошибки, вводят переменные ξ_i (slack variables) и параметр регуляризации C :

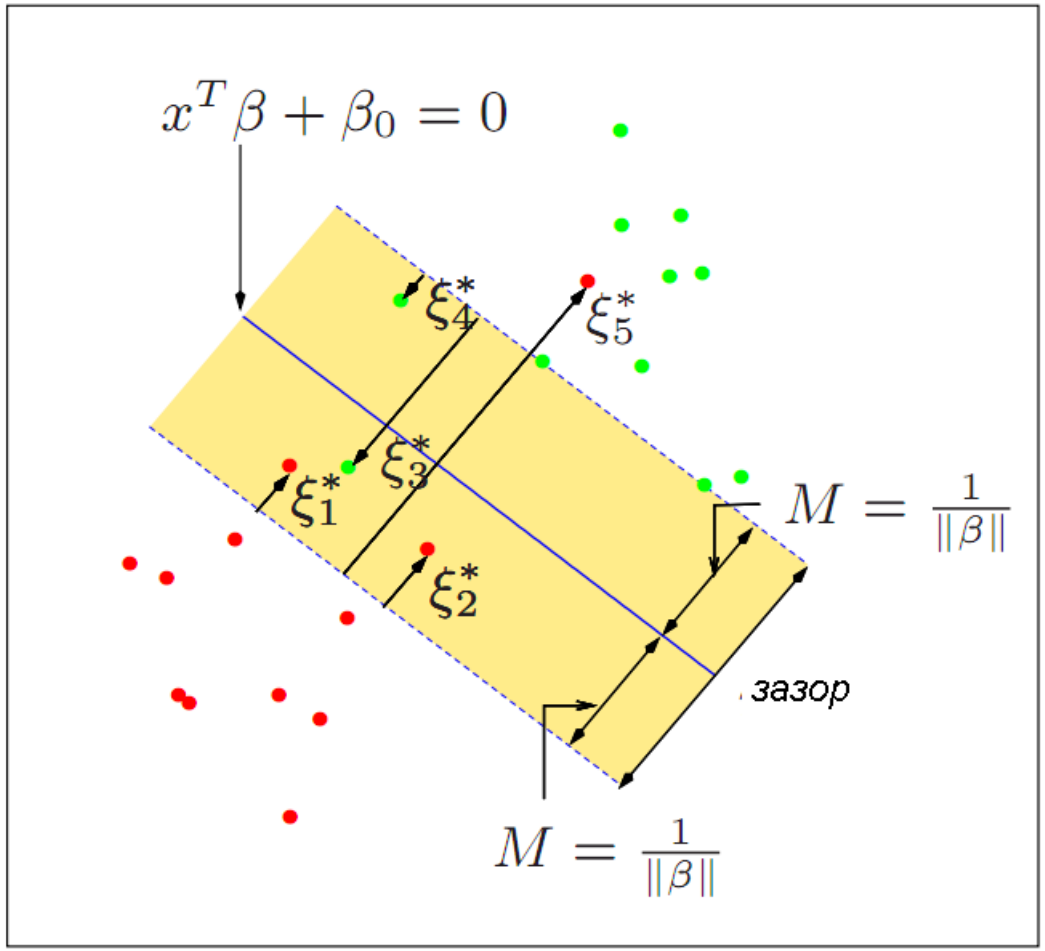
$$\min_{w,b} \frac{1}{2}\|w\|^2 + \mathrm{C}\sum_{i=1}^{n} \xi_i$$

- C большой: алгоритм штрафует за любую ошибку (жесткая граница).
- C маленький: алгоритм допускает ошибки ради более широкого зазора (мягкая граница).

Алгоритм SVM

4. Мягкий зазор (Soft Margin)





Алгоритм SVM

5. Переход в высшее пространство (Kernel Trick)

Если данные «линейно неразделимы» (например, один круг внутри другого), алгоритм применяет математическую функцию — ядро. Оно переносит данные в новое измерение (например, из 2D в 3D), где их можно легко разделить одной ровной плоскостью.

Функции ядер

Когда данные нельзя разделить прямой линией или плоскостью в их текущем виде, на помощь приходят **функции ядер (Kernels)**.

Их магия в том, что они вычисляют «близость» точек в пространстве более высокой размерности, не выполняя само преобразование координат (это экономит кучу вычислительных ресурсов).

Линейное ядро (Linear Kernel)

Это самый простой вариант. Оно не меняет размерность и просто ищет прямую линию.

Формула:

$$K(x, x_i) = \sum_{i=1}^{n} (x \cdot x_i)$$

Когда использовать: Если у вас очень много признаков (например, классификация текстов), данные часто уже разделимы линейно.

Полиномиальное ядро (Polynomial Kernel)

Оно представляет данные в виде комбинаций исходных признаков (возводит их в степень). Представьте, что вы искривляете пространство, чтобы «обернуть» границу вокруг группы точек.

Формула:

$$K(x, x_i) = \sum_{i=1}^{n} (x \cdot x_i + r)^d$$

Параметры: $\boldsymbol{d}$ — степень полинома. Чем выше d, тем сложнее (извилистее) граница.

Когда использовать: В задачах распознавания образов.

RBF (Radial Basis Function) / Гауссово ядро

Самое популярное и мощное ядро «по умолчанию». Оно работает так, будто вокруг каждой точки создается «облако влияния». Оно теоретически переносит данные в пространство бесконечной размерности.

Формула:

$$K(x, x_i) = e^{-\gamma \|x - x_i\|^2}$$

Параметры:

- **Большая γ** — модель учитывает только ближайшие точки (граница будет очень «дерганой»).
- **Маленькая γ** — модель смотрит на данные более глобально (граница будет плавной).

Когда использовать: Когда вы не знаете, какую форму имеет граница между классами.

Сигмоидное ядро (Sigmoid Kernel)

По принципу работы оно похоже на функцию активации в нейронных сетях.

Формула:

$$K(x, x_i) = \tanh(\alpha x^T x_i + c)$$

Когда использовать: Пришло из теории нейросетей, но на практике в SVM используется реже, чем RBF.

Сравнение, выбор ядра

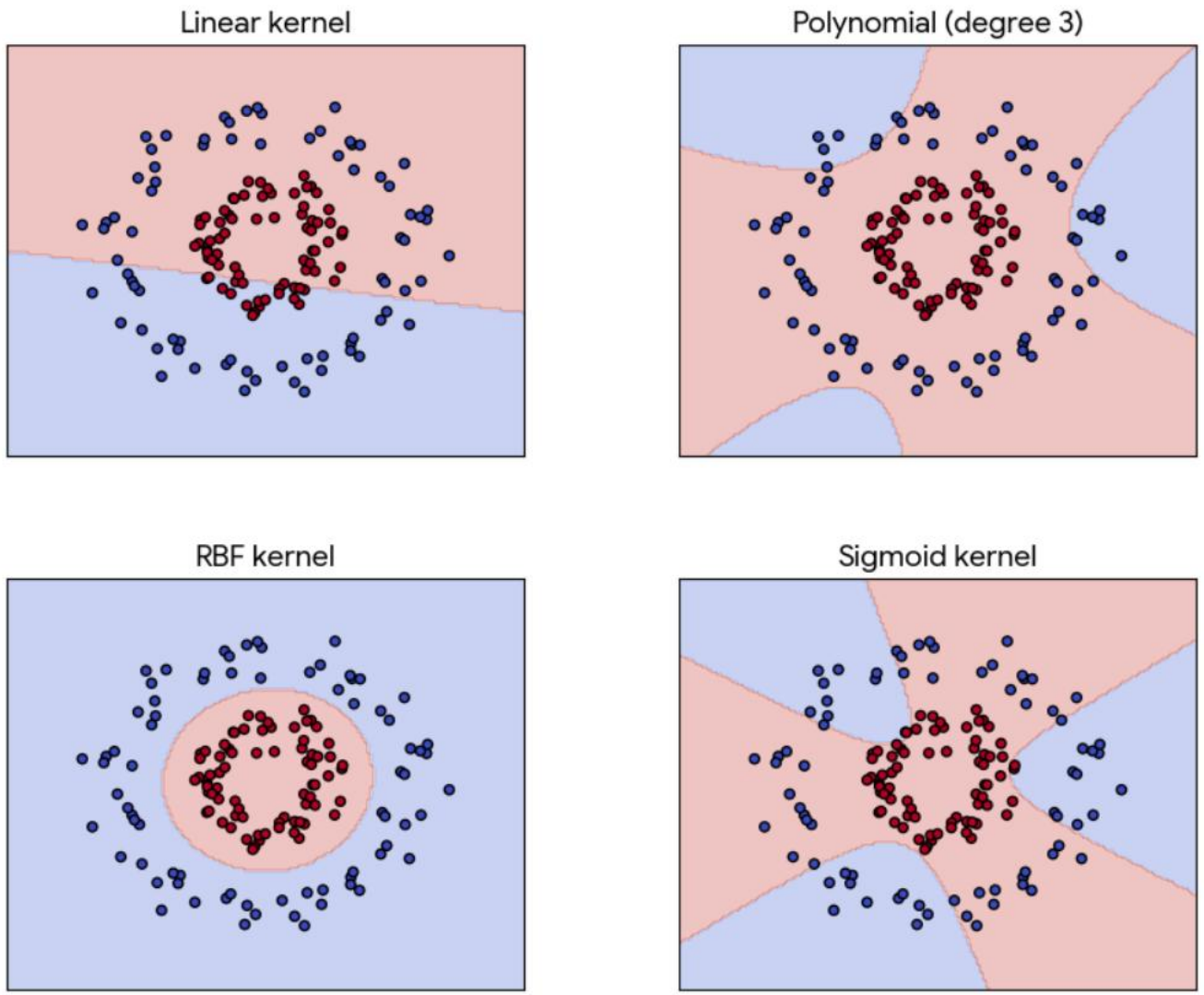
- ✓ Начните с линейного, если признаков больше, чем объектов.
- ✓ Попробуйте RBF, если признаков мало, а объектов много.
- ✓ Полиномиальное стоит пробовать, если есть подозрение на конкретные зависимости признаков.

Чтобы наглядно увидеть разницу, представим сложный случай: **один класс данных находится внутри другого**. Обычная прямая линия здесь бессильна.

Сравнение, выбор ядра

- **Linear (Линейное):** Пытается провести прямую черту. Как видно, оно полностью проваливает задачу, так как данные имеют круговую структуру. Синие точки внутри красного кольца остаются неразбитыми.
- **Polynomial:** За счет возведения признаков в степень граница начинает изгибаться. Однако при степени $d = 3$ оно все еще может не идеально описывать круг, создавая более сложные, но не всегда симметричные формы.
- **RBF (Гауссово):** Идеально справляется с задачей. Оно создает замкнутую область вокруг центрального кластера. Это ядро наиболее адаптивно и «обволакивает» данные любой формы.
- **Sigmoid (Сигмоидное):** Пытается имитировать поведение нейронных сетей. В данной задаче оно часто ведет себя непредсказуемо и редко дает хорошую границу для круговых данных.

Сравнение, выбор ядра



Подбор параметра Gamma (γ)

Параметр Gamma (γ) в SVM с ядром RBF определяет «радиус влияния» отдельного опорного вектора. Проще говоря, он говорит алгоритму, насколько далеко распространяется «авторитет» одной точки данных.

Маленькая γ:

- **Радиус:** Очень большой. Каждая точка влияет на классификацию точек, находящихся далеко от неё.
- **Граница:** Получается очень плавной, почти прямой.
- **Риск: Недообучение (Underfitting).** Модель может пропустить важные изгибы в данных и стать слишком простой.

Подбор параметра Gamma (γ)

Параметр Gamma (γ) в SVM с ядром RBF определяет «радиус влияния» отдельного опорного вектора. Проще говоря, он говорит алгоритму, насколько далеко распространяется «авторитет» одной точки данных.

Большая γ:

- **Радиус:** Маленький. Точка влияет только на ближайших «соседей».
- **Граница:** Становится очень сложной, извилистой, буквально «облегает» каждую точку.
- **Риск: Переобучение (Overfitting).** Модель выучит шум и случайные колебания в данных вместо общей закономерности.

Как подбирать γ на практике?

Подбор γ всегда идет в связке с параметром C (штраф за ошибки). Основной инструмент для этого — **Grid Search (Поиск по сетке) с кросс-валидацией.**

- **Логарифмическая сетка:** γ обычно меняется в геометрической прогрессии. Начните с широкого диапазона: $10^{-4}, 10^{-3}, 10^{-2}, \dots, 10^{2}, 10^{3}$.
- **Визуализация валидации:** Используйте тепловую карту (Heatmap) точности, где по одной оси — C, по другой — γ. Ищите «пятно», где точность на тесте максимальна.
- **Правило «золотой середины»:** Если граница на графике выглядит как рваные лоскуты вокруг каждой точки — γ слишком велика. Если она почти не гнется там, где это явно нужно — γ слишком мала.

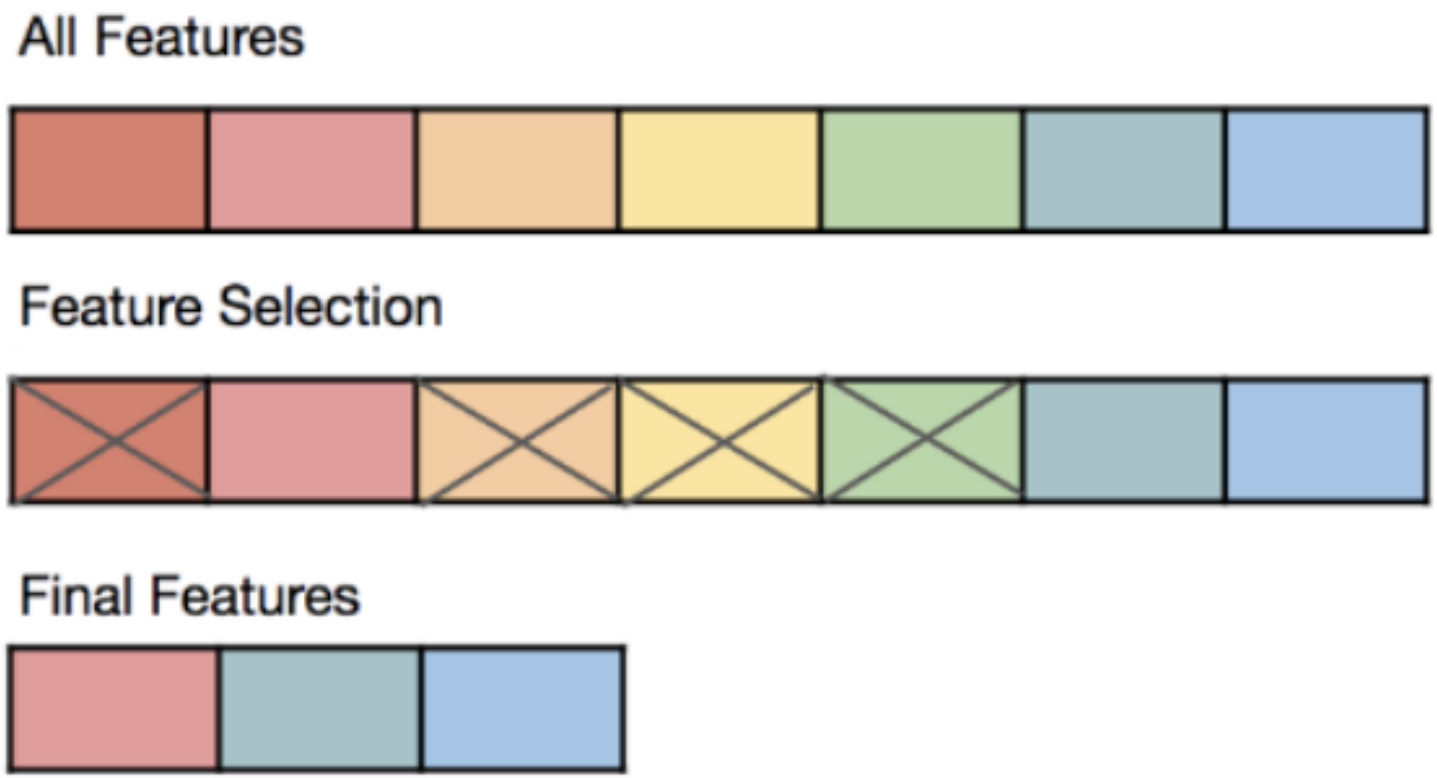
Практические аспекты

Понижение размерности пространства признаков

t-SNE/UMAP

PCA

Понижение размерности пространства признаков



Отбор признаков (Feature Selection)

Здесь мы просто выкидываем лишние столбцы, оставляя оригинальные данные нетронутыми.

- **L1-регуляризация (Lasso):** Добавляя штраф за модуль весов в функции потерь, Lasso буквально «обнуляет» коэффициенты у самых слабых признаков. Если коэффициент стал 0 — признак можно удалять.
- **Статистические фильтры:** Мы считаем корреляцию признаков с целевой переменной или их взаимную корреляцию. Если два признака коррелируют на 0.99, один из них лишний.
- **Жадные алгоритмы (RFE):** Рекурсивное исключение признаков. Обучаем модель, смотрим на важность признаков, выкидываем самый слабый, повторяем до нужного количества.

Извлечение признаков (Feature Extraction)

Здесь создаётся смесь из старых признаков, дающая новые оси. К этой группе относится PCA (с которым мы сегодня познакомимся).

Линейные методы:

- **LDA (Linear Discriminant Analysis):** Похож на PCA, но учитывает «метки» классов. Он ищет такие оси, чтобы точки одного класса были максимально близко, а разных — максимально далеко.
- **ICA (Independent Component Analysis):** Пытается разделить смешанные сигналы на независимые источники (например, выделить голос одного человека из шума толпы).

Извлечение признаков (Feature Extraction)

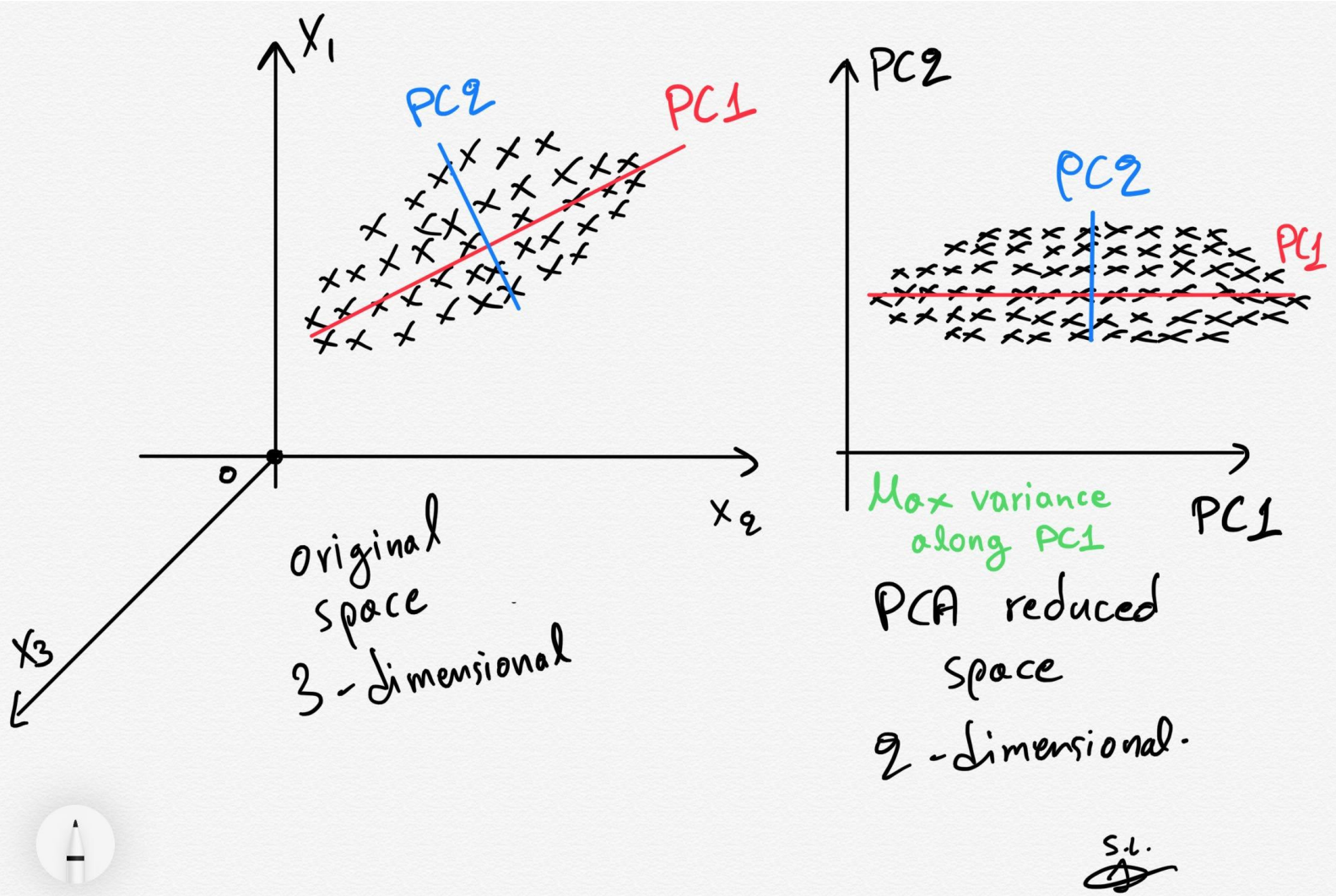
Нелинейные методы (Manifold Learning):

- **t-SNE и UMAP:** Короли визуализации. Они пытаются сохранить «соседство» точек: если в 100D объекты были похожи, в 2D они тоже будут рядом. Но их сложно использовать для обучения моделей, так как нет обратного преобразования.
- **Autoencoders (Автокодировщики):** Нейросети, которые сжимают данные в узкое «горлышко» (энкодер) и пытаются восстановить их обратно (декодер). Средний слой и есть сжатое признаковое пространство.

Выбор метода

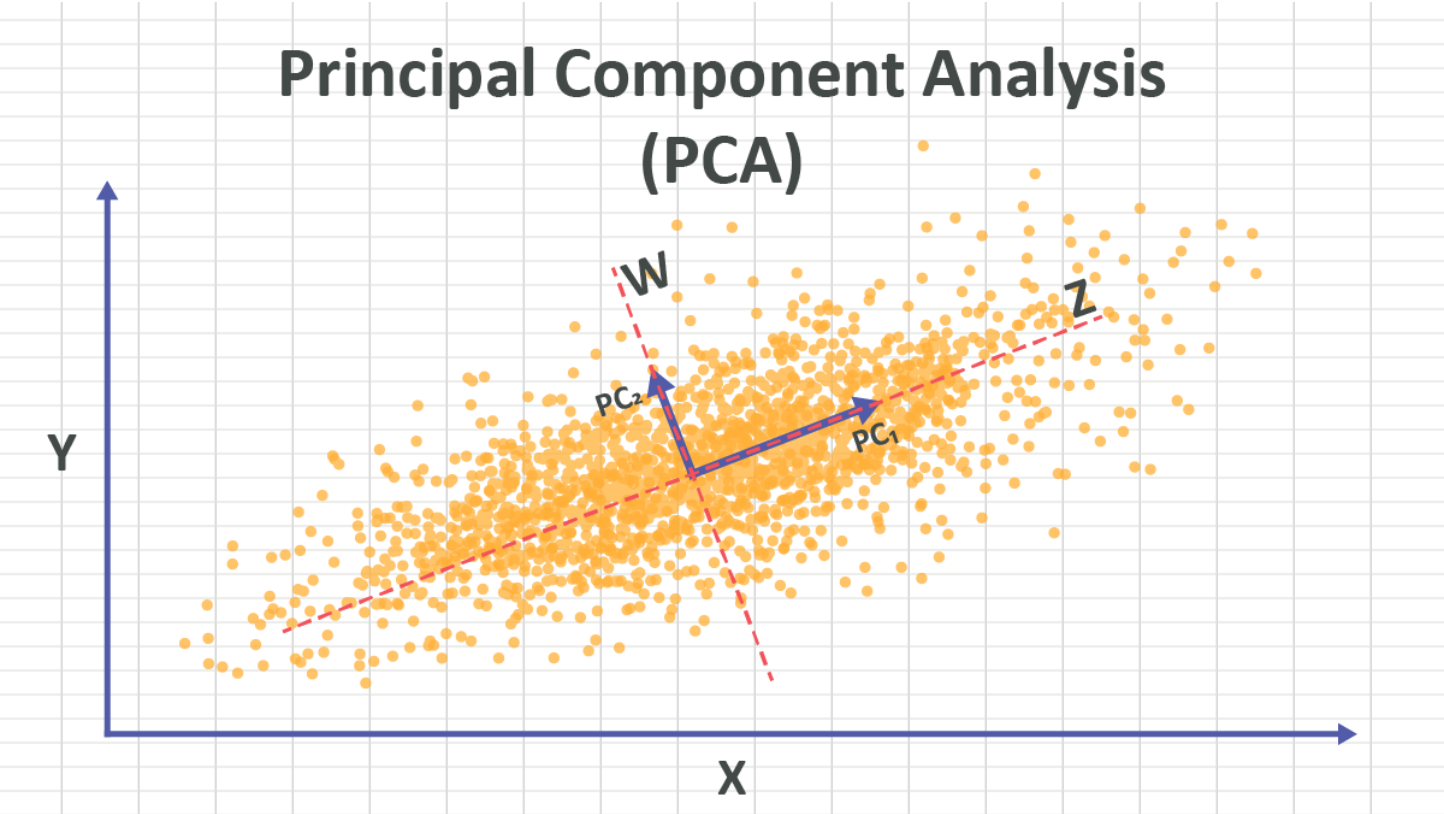
- Если нужно понимать физический смысл каждого признака — используем L1 или RFE.
- Если нужно максимально сжать данные для модели, не теряя информацию — используем PCA или Autoencoders.

Понижение размерности пространства признаков



PCA (Principal Component Analysis)

Метод главных компонент (PCA) — это способ упростить данные, сохранив в них самое важное. Представим облако точек: PCA находит такие направления (оси), вдоль которых данные «растянуты» сильнее всего.



Метод главных компонент (Principal Component Analysis, PCA)

- **Метод главных компонент (Principal Component Analysis, PCA) —** это статистический метод обучения без учителя, используемый для снижения размерности данных с минимальной потерей информации. Он преобразует коррелированные признаки в новые, некоррелированные «главные компоненты», максимизируя дисперсию, что упрощает визуализацию, очищает данные от шума и ускоряет машинное обучение.

Как работает PCA

- **Центрирование:** Мы сдвигаем данные так, чтобы их среднее было в нуле.
- **Поиск направлений:** Первая главная компонента (PC_1) ложится вдоль самого длинного участка данных (максимальная дисперсия). Вторая (PC_2) идет перпендикулярно первой и забирает оставшуюся часть изменчивости.
- **Проекция:** Мы выбрасываем малозначимые компоненты (где данных почти нет) и проецируем точки на оставшиеся оси.
- **Итог:** Было 100 признаков, осталось 3, но они описывают 95% сути.

Связь PCA с SVD (Сингулярным разложением)

- SVD — это «двигатель» под капотом PCA. Если PCA — это концепция того, *что* мы хотим сделать, то SVD — это универсальный алгоритм, который это *вычисляет*.
- Математически PCA чаще всего реализуется именно через SVD. Это более стабильный и быстрый способ, чем работа через матрицу ковариации.
- SVD разлагает любую матрицу $\boldsymbol{X}$ объектов-признаков на три части:

$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\boldsymbol{T}}$$

Связь PCA с SVD (Сингулярным разложением)

$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\boldsymbol{T}}$$

- $\boldsymbol{V}$(Правые сингулярные векторы): Это и есть направления наших главных компонент. Они показывают, как старые признаки складываются в новые.
- $\boldsymbol{\Sigma}$(Сингулярные числа): Показывают «силу» каждой компоненты. С их помощью мы понимаем, сколько информации потеряем, если удалим лишние измерения.
- $\boldsymbol{U}\boldsymbol{\Sigma}$: Это координаты ваших данных в новом, сокращенном пространстве.

t-SNE (t-distributed Stochastic Neighbor Embedding)

Главный принцип: «соседи в многомерном пространстве должны остаться соседями на плоскости».

Как это работает: Алгоритм считает вероятность того, что точки являются соседями. Затем он «расталкивает» точки на плоскости так, чтобы похожие оказались рядом, а непохожие — далеко.

Главная фишка: Очень хорошо разделяет данные на четкие кластеры.

t-SNE. Недостатки

- **Медленный** (на больших данных работает долго).
- **Не сохраняет глобальную структуру:** Расстояния между кластерами на графике могут ничего не значить.
- **Результат зависит от Perplexity:** Это параметр «количества соседей», который нужно подбирать вручную.

UMAP (Uniform Manifold Approximation and Projection)

- **Как это работает:** Он строит своего рода «граф» или каркас данных в высоком измерении и пытается максимально точно перенести этот каркас на плоскость.
- **Главная фишка:** Он **быстрее** и лучше сохраняет **глобальную структуру** (то есть не только точки внутри кластера близки, но и сами кластеры располагаются логично относительно друг друга).

UMAP (Uniform Manifold Approximation and Projection). Преимущества

- Работает значительно быстрее t-SNE.
- Масштабируется на очень большие датасеты.
- Позволяет добавлять новые точки в уже построенную проекцию (t-SNE так не умеет).

Обучение без учителя

Постановка задачи кластеризации

K-means, алгоритм кластеризации Ллойда

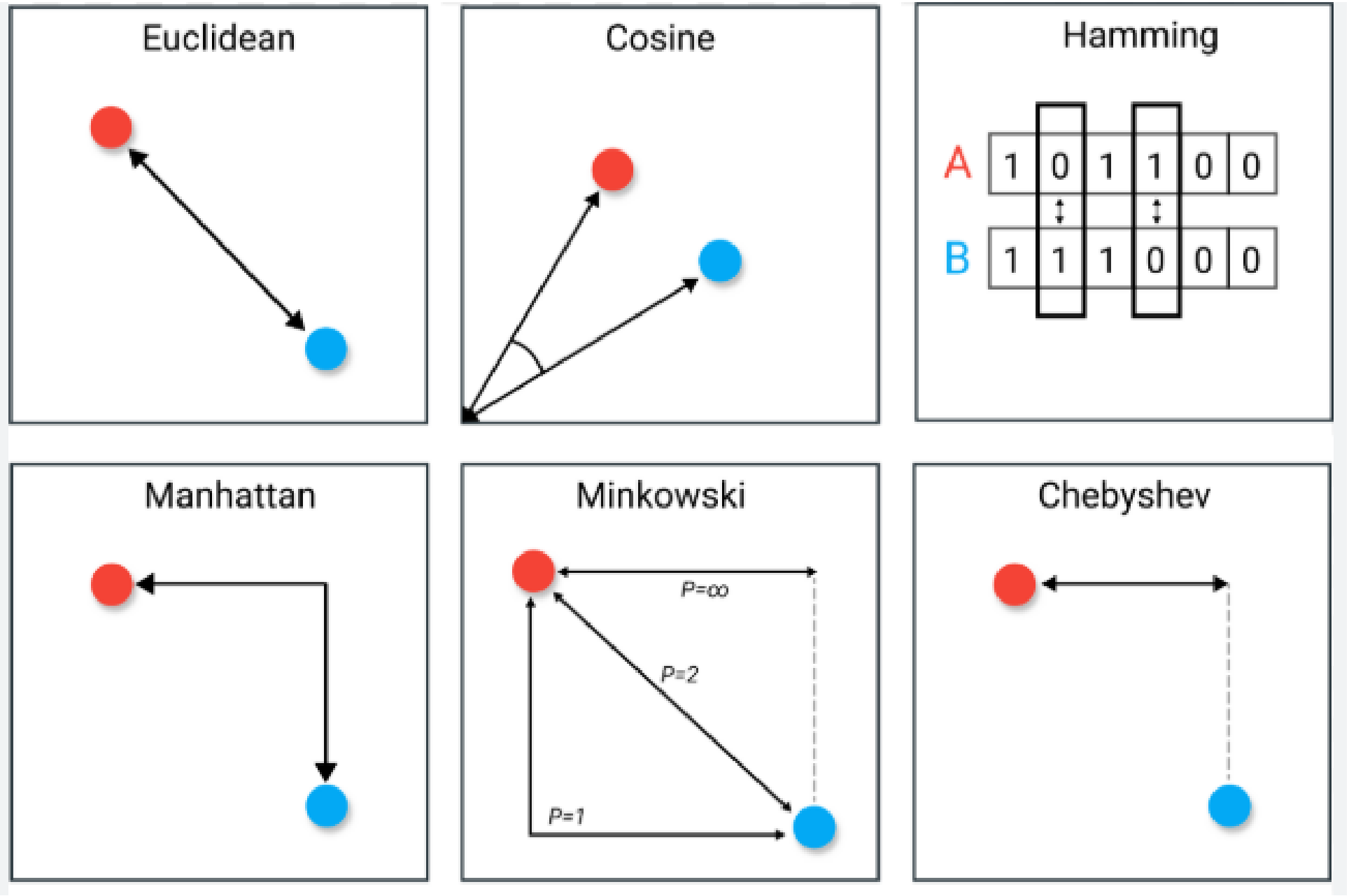
Алгоритм кластеризации DBSCAN

Задача кластеризации

- **Кластеризация** — это задача машинного обучения «без учителя», когда нужно объединить объекты в группы (кластеры) так, чтобы похожие данные оказались вместе, а разные — как можно дальше друг от друга.
- Главная фишка здесь в том, что нет заранее известных ответов (меток). Алгоритм сам ищет скрытые закономерности в структуре данных.

Постановка задачи кластеризации

- **Входные данные:** У нас есть множество объектов, каждый из которых описан набором признаков (числа, категории).
- **Мера сходства:** Нужно определить, что такое «похожесть». Обычно это расстояние между точками в пространстве (например, евклидово расстояние). Чем меньше расстояние, тем больше сходства.
- **Цель – распределить объекты по группам так, чтобы:**
  - Внутрикластерное сходство было максимальным (точки в одном круге сидят плотно).
  - Межкластерное сходство было минимальным (сами группы стоят далеко друг от друга).


Euclidean
Cosine
Hamming
A
1 0 1 1 0 0
B
1 1 1 0 0 0
Manhattan
Minkowski
P=∞
P=2
P=1
Chebyshev

Применение

- **Сегментация клиентов:** Разбить покупателей на группы по поведению, чтобы делать точные рекламные предложения.
- **Сжатие изображений:** Уменьшить количество цветов, объединив похожие оттенки в один.
- **Анализ текстов:** Группировка новостей по темам без ручного чтения.
- **Биология:** Классификация генов или видов животных по признакам.

Метод k-means

Если говорить глобально, K-means (K-средних) — это итеративный алгоритм, который разбивает данные на k заранее заданных групп. Его главная цель — минимизировать суммарное квадратичное отклонение точек кластера от их центров.

Метод k-means

- **Параметр K:** Это «магическое число», которое мы задаем сами. Алгоритм не знает, сколько групп в данных, он просто выполняет приказ: «Раздели это на k частей».
- **Центроиды:** Это «штаб-квартиры» кластеров. В начале они ставятся случайно, а в конце — оказываются в самом геометрическом центре своей группы точек.
- **Евклидово расстояние:** Самый частый способ измерить близость. Алгоритм просто тянет линейку от точки до всех центроидов и выбирает кратчайший путь.

Алгоритм Ллойда

Алгоритм Ллойда — это классический и самый популярный способ реализации метода K-means (K-средних).

Преимущества:

- **Скорость:** Он работает очень быстро даже на больших объемах данных.
- **Простота:** Его легко понять и запрограммировать.

Алгоритм Ллойда

Недостатки:

- **Рандомизация решает всё:** Если в самом начале поставить центроиды неудачно, алгоритм может застрять в «плохом» варианте. Поэтому часто его запускают несколько раз.
- **Только сферы:** Алгоритм Ллойда хорошо ищет только округлые, выпуклые группы. Если данные лежат «бумерангом» или «змейкой», он их адекватно не разделит.
- **Чувствительность к выбросам:** Одна точка очень далеко в стороне может сильно «утянуть» центр кластера на себя.

Алгоритм Ллойда

1. Инициализация: Выбор числа k (сколько групп хотим получить) и ставим в пространстве данных k случайных точек. Это — начальные центроиды (центры масс будущих кластеров).

2. Шаг привязки (Assignment): Каждый объект из данных «смотрит», какой из центроидов к нему ближе всего (обычно по прямой — Евклидову расстоянию), и примыкает к нему. Так формируются временные кластеры.

3. Шаг обновления (Update): Внутри каждой полученной группы вычисляется «центр тяжести» (среднее арифметическое всех координат точек). Старый центроид прыгает в это новое место.

4. Повторение: Шаги 2 и 3 повторяются снова и снова. Центроиды двигаются, точки меняют владельцев, пока положение центров не стабилизируется (они перестанут двигаться).

Алгоритм DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) расшифровывается как «кластеризация, основанная на плотности». Его главная фишка: он не просит угадывать число кластеров, а сам находит области, где точки сидят «густо».

Алгоритм ищет произвольную ядровую точку и рекурсивно присоединяет к ней все плотностно-достижимые точки, формируя кластер. Если точка не достижима ни из одного ядра, она помечается как шум.

Алгоритм DBSCAN

1. **Основные определения:**

Для набора точек D вводятся следующие понятия:

- **$\boldsymbol{\varepsilon}$-окрестность точки $\boldsymbol{p}$**: Множество точек $N_\varepsilon(p) = \{q \in D | dist(p, q) \leq \varepsilon\}$
- **Ядровая точка:** точка p, у которой количество соседей не меньше заданного порога $|N_\varepsilon(p)| \geq MinPts$
- **Достижимая по плотности точка:** Точка q достижима из p, если p – ядровая точка и $q \in N_\varepsilon(p)$
- **Связность по плотности:** Точки p и q связны, если существует цепочка точек $p_{1,}, p_2, \dots, p_n$, где $p_1 = p, p_n = q$ и каждая последующая точка достижима из предыдущей.

Алгоритм DBSCAN

2. Формализация кластера:

Кластер C в DBSCAN – это непустое подмножество точек, удовлетворяющее двум условиям:

- **Максимальность:** Если $\mathrm{p} \in C$ и q достижима из p по плотности, то $q \in C$
- **Связность:** Все точки внутри кластера C плотно-связны между собой.

Алгоритм DBSCAN

3. Классификация точек:

После работы алгоритма каждая точка попадает в одну из трёх категорий:

- Ядро: внутри плотной области.
- Граничная: входит в окрестность ядра, но сама не имеет достаточно соседей.
- Шум: Точки, не попавшие ни в одну ε-окрестность ядровых точек, т. е. $D \setminus C_i$.

Алгоритм DBSCAN

Шаг 1. Помечаем все точки как непосещенные.

Шаг 2. Выбираем случайную точку p. Если она уже посещена, переходим к следующей.

Шаг 3. Проверяем условие плотности: Находим все точки в $N_\varepsilon(p)$. Если $|N_\varepsilon(p)| < MinPts$, то помечаем p как шум, иначе создаём новый кластер C и включаем в него p.

Шаг 4. Расширяем кластер:

4.1. Добавляем все точки из $N_\varepsilon(p)$ в очередь расширения S.

Алгоритм DBSCAN

4.2. Для каждой точки $q \in S$:

- Если q была помечена как шум, меняем статус на «граничная точка» и добавляем в C.
- Если q не посещена, помечаем как посещенную. Если при этом q – основная точка ($|N_\varepsilon(q)| \geq MinPts$), добавляем все точки из $N_\varepsilon(q)$, в очередь расширения S.
- Добавляем q в кластер C.

Шаг 5. Повторяем, пока не закончатся непосещенные точки в наборе данных.

Алгоритм Ллойда vs DBCSAN

Чтобы наглядно понять разницу, представим, что мы смотрим на карту города сверху:

- **K-means** — это как деление города на административные районы
- **DBSCAN** — это поиск реальных скоплений людей на улицах. Мы не задаём заранее, на сколько скоплений нам нужно разделить, только «правила плотности» (расстояние и минимум соседей). Алгоритм сам скажет: «Я нашел здесь 5 групп, а всё остальное — мусор».

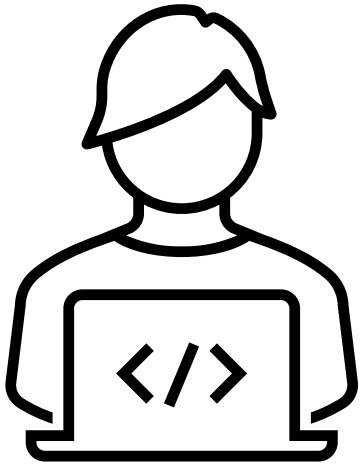
Алгоритм Ллойда vs DBCSAN. Выбросы в данных

- **K-means:** Каждую точку обязан пристроить хоть к какому-то кластеру. Если одна точка улетела «в космос», она всё равно будет числиться в ближайшем кластере и, скорее всего, сильно сместит его центр к себе.
- **DBSCAN:** Очень «придирчивый». Если точка стоит в стороне и у неё нет компании, он пометит её как Noise (шум). Это идеальный способ чистить данные от аномалий.

Алгоритм Ллойда vs DBCSAN. Плотность в данных

- **K-means:** Хорошо работает, даже если плотность в разных группах разная.
- **DBSCAN:** Пасует, если в данных есть «густые» города и «редкие» деревни. Если ты настроишь его на поиск городов, он проигнорирует деревни как шум. Если настроишь на деревни — города сольются в одно огромное пятно.

Практика



Ресурсы для изучения ML

Лабораторные работы